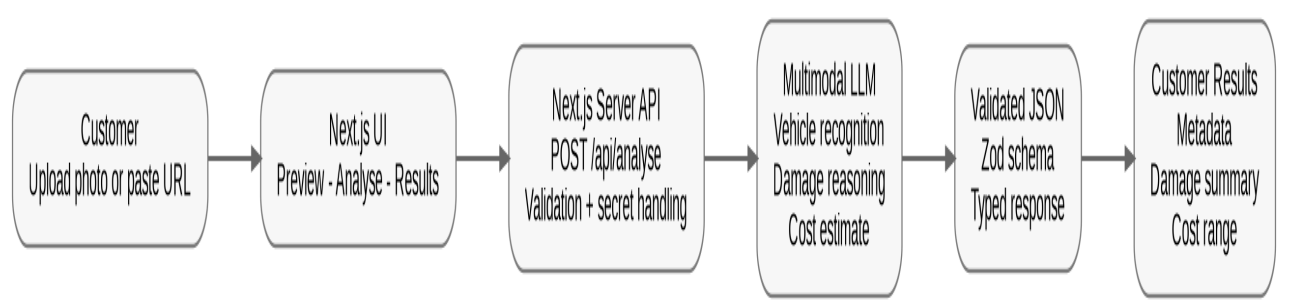


AI Vehicle Damage Prototype

Architecture and Response Data Model - Design Decisions

Purpose: establish the MVP contract before coding so that the front end, API, multimodal model, validation layer and future evaluation framework all share one clear data model.

Scope anchor. The prototype requires a minimal full-stack application that accepts a damaged-car photograph or URL and returns vehicle metadata, a damage summary and a rough AI-generated repair cost. The design below adds only fields that materially improve assessor trust or future extensibility.



MVP architecture choice

Layer	Choice	Why this choice now
Application	Next.js + TypeScript	One repository for UI and server API; fastest path to an end-to-end prototype.
UI	React + Tailwind	Simple upload/URL workflow, preview, loading/error states and result cards.
Server	POST /api/analyse	Keeps model credentials server-side and gives one stable integration contract.
AI	One multimodal foundation model	Fastest way to validate vehicle recognition + damage reasoning + rough cost estimation.
Validation	Zod / typed schema	Rejects malformed model output before the UI consumes it.
Persistence	None in MVP	Not required to prove the user journey; avoids unnecessary privacy/governance scope.

1. Response / Data Model Decision

Your proposed additions - **confidence**, **severity**, **affectedAreas**, **costRange** and **assumptions** - should be included. They make the output more useful to an assessor and create hooks for later evaluation and human-review workflows.

Important caveat on confidence: an LLM self-reported confidence score is a useful UI signal, but it is not automatically a calibrated probability. For a production claims system, confidence thresholds should be calibrated against labelled evaluation data and tied to human-review rules.

Field	MVP?	Suggested shape	Why it matters
vehicle.make / model / color	Yes	string	Required by brief; core identification output.
vehicle.confidence	Yes	0-1 or low/medium/high	Signals uncertainty in identification; should be labelled as model confidence.
damage.summary	Yes	string	Required by brief; concise customer-readable description.
damage.severity	Yes	minor moderate severe unknown	Useful triage signal and later routing criterion.
damage.affectedAreas	Yes	string[]	Makes the assessment structured and easier to evaluate/filter.
damage.confidence	Yes	0-1 or low/medium/high	Separates confidence in damage interpretation from vehicle identification.
repairEstimate.costRange	Yes	min, max, currency	More honest than a falsely precise single cost.
repairEstimate.assumptions	Yes	string[]	Explains what the estimate depends on and exposes uncertainty.
repairEstimate.confidence	Recommended	0-1 or low/medium/high	Cost is the least grounded output; useful to signal additional uncertainty.
reviewRequired	Recommended	boolean	Creates a clean bridge to human-in-the-loop claims handling.
warnings	Recommended	string[]	Surfaces issues such as poor image quality or possible hidden damage.

Recommended MVP JSON contract

```
{
  "vehicle": {
    "make": "BMW", "model": "3 Series", "color": "Black",
    "confidence": 0.84
  },
  "damage": {
    "summary": "Dent and abrasion to left rear bumper",
    "severity": "moderate",
    "affectedAreas": ["rear bumper", "left quarter panel"],
    "confidence": 0.88
  },
  "repairEstimate": {
    "costRange": {"min": 900, "max": 1400, "currency": "GBP"},
    "assumptions": ["No structural damage visible", "Estimate uses visible damage only"],
    "confidence": 0.58
  },
  "reviewRequired": true,
  "warnings": ["Final repair cost requires professional inspection"]
}
```

2. Alternatives Considered

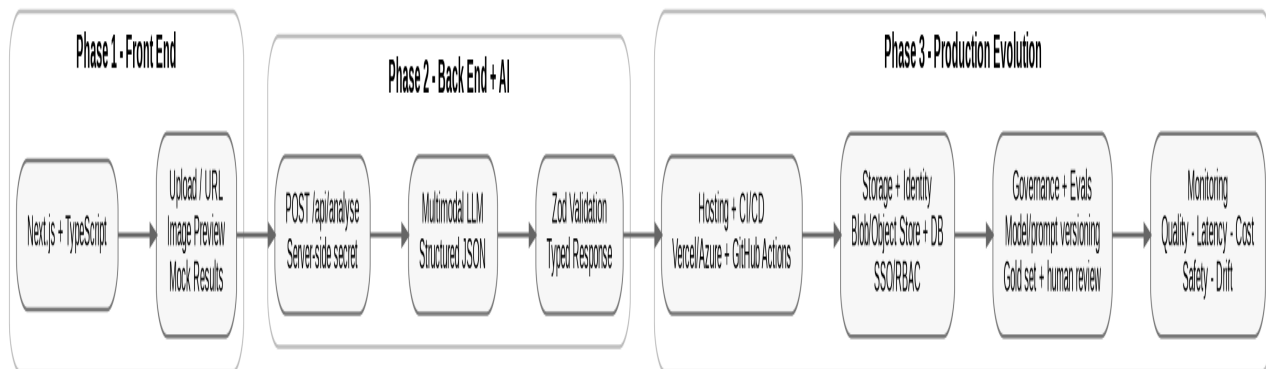
Decision area	Chosen for MVP	Alternative	Why not now / when to revisit
Cost output	Range + assumptions	Single point estimate	Single values imply false precision. Revisit only if backed by deterministic pricing data.
Confidence	Per-section signal	One global confidence	A single number hides which part is uncertain. Per-section confidence is more actionable.
Damage structure	Summary + severity + areas	Free-text summary only	Structured fields improve evaluation, routing and future analytics with little extra effort.
Image evidence	Textual affected areas	Bounding boxes / segmentation masks	Visually compelling but adds model/tooling complexity. Strong Phase 3 enhancement.
Pricing logic	LLM-generated rough range	Parts/labour pricing engine	Production should use repair data, vehicle parts and labour rates; not needed for prototype.
Persistence	Stateless	Blob/object storage + claims DB	History and authentication are intentionally deferred to a later production phase.
Architecture	Single Next.js app	Separate React + FastAPI services	More components, deployment and integration overhead without improving prototype value.
AI orchestration	Single multimodal call	Multi-model / agent pipeline	Use only if evals show specialist models materially improve accuracy.

Additional fields worth planning now, but not necessarily showing in the MVP UI

- **assessmentId** - useful once assessments are persisted or referenced externally.
- **modelVersion / promptVersion** - essential for production traceability and regression analysis.
- **processingTimeMs** - operational telemetry for latency monitoring.
- **inputQuality** - e.g. good / poor / unusable; useful when image quality drives uncertainty.
- **recommendedAction** - e.g. continue, request another photo, human review. Better added once routing rules are defined.

Recommendation: keep these as internal metadata or Phase 3 fields. Do not overload the customer-facing prototype with operational details.

3. Three-Phase Architecture Evolution



Phase	Goal	What gets built	Exit criterion
Phase 1 - Front End	Prove the customer journey	Next.js UI; upload/URL; image preview; loading/error states; result cards using mock JSON.	A user can complete the entire flow using mocked results with no backend dependency.
Phase 2 - Back End + AI	Make the prototype functional	POST /api/analyse; image validation; multimodal model call; structured output; Zod validation; real results returned to UI.	Uploaded image produces valid vehicle, damage and cost-range results end to end.
Phase 3 - Production Evolution	Show enterprise readiness without overbuilding the prototype	Hosting + CI/CD; storage/history; identity/RBAC; model/prompt versioning; eval set; human review; monitoring; governance.	Architecture and SOW explain how the prototype becomes a trusted production claims capability.

Design principle

Prototype for speed; architect for trust. The live implementation should stay minimal, while the architecture and SOW show how confidence calibration, human review, repair-pricing data, persistence, identity, monitoring and evaluations would be introduced before using the system for consequential claims decisions.

Next logical step

Freeze the TypeScript/Zod schema above as the API contract, then define the exact request/response behaviour for **POST /api/analyse**. Only after that should GitHub Copilot scaffold the Phase 1 front end. This keeps Copilot as the implementation accelerator while the product and architecture decisions remain explicit and explainable.